TransitFlow — dokumentacija sistema preporuke

1. Šta sistem preporučuje i gdje se koristi

Sistem preporučuje transportne linije (TransportLine) koje bi korisniku mogle biti interesantne.

Backend:

- Endpoint: GET /api/recommendations/lines?count=N
- Feedback: POST /api/recommendations/feedback
- Provjera feedbacka: GET /api/recommendations/feedback/{transportLineId}

Mobilna aplikacija:

- Ekran: "Preporučene linije" (RecommendedLinesScreen)
- Na ekranu postoji opcija "Kako funkcioniše sistem preporuke?" i pojedinačna objašnjenja po liniji (dialog "Objašnjenje preporuke").

2. Koji podaci/signali ulaze u preporuku?

Sistem koristi stvarne podatke koje korisnik generiše u aplikaciji:

a) Historija kupovine karata (Tickets)

- Gleda se zadnja 3 mjeseca.
- Iz karte se uzima ruta, a iz rute transportna linija.
- Broj kupljenih karata po liniji se koristi kao signal interesa.

b) Omiljene linije (FavoriteLines)

- Linije koje je korisnik označio kao omiljene povećavaju skor.

c) Feedback na preporuke (RecommendationFeedbacks)

- Pozitivan feedback (korisno) snažno povećava skor.
- Negativan feedback (nekorisno) se koristi kao "blocklist" — linija se preskače i ne preporučuje.

d) Sličnost sa drugim korisnicima (collaborative)

- Poredi se skup linija koje koristi trenutni korisnik sa skupom linija drugih korisnika.
- Koristi se Jaccard sličnost: $|A \cap B| / |A \cup B|$.

Napomena: A i B predstavljaju skup različitih linija koje korisnik koristi (prisustvo/odsustvo linije), a ne broj vožnji ili broj kupljenih karata.

3. Opis algoritma

Algoritam je hibridan: kombinuje korisničke signale (kupovine, favorite, feedback) i doprinos "sličnih korisnika".

Koraci:

1. Učita sve aktivne linije.
2. Učita korisnikove karte iz zadnja 3 mjeseca, favorite, pozitivne i negativne feedbackove.
3. Formira skup "linija korisnika" (linije iz kupovina i favorita).
4. Cold-start fallback:

   Ako korisnik nema nijednu liniju (nema kupovina/favorita) i nema pozitivnog feedbacka, sistem vraća početnu listu aktivnih linija uz objašnjenje "Početna preporuka – još nema dovoljno podataka".
5. Za napredniji dio:
   - Učita karte/favorite/pozitivne feedbackove i za ostale korisnike (zadnja 3 mjeseca).
   - Formira "rating" po korisniku i liniji:
     - kupovina karte: +1.0 po karti
     - dodavanje u omiljene: +3.0
     - pozitivan feedback: +2.0
   - Rating se koristi za slične korisnike, a osnovni doprinos za trenutnog korisnika; zato vrijednosti nisu iste.
   - Računa sličnost trenutnog korisnika sa drugim korisnicima (Jaccard).
6. Računanje skora po liniji (za svaku aktivnu liniju):

- Ako je linija u negativnom feedbacku: preskače se.
- Osnovni doprinos (direktno ponašanje korisnika):
  - kupovine: purchasedCount * 1.0
  - omiljena linija: +5.0
  - pozitivan feedback: +10.0
- Doprinos sličnih korisnika:
  - za svakog sličnog korisnika koji ima rating za tu liniju: score += similarity(otherUser) * rating(otherUser, line)

7. Linije se sortiraju po skoru (opadajuće) i uzima se top N.
8. Ako nema dovoljno linija sa skorom, popuni se do N dodatnim aktivnim linijama (koje nisu u negativnom feedbacku).

4. Objašnjive preporuke

Svaka preporučena linija može dobiti tekstualno objašnjenje (scoreExplanation) koje korisnik može otvoriti na mobilnoj aplikaciji.

Primjeri objašnjenja:

- "Označili ste kao korisno (+10.0 bonus)"
- "U vašim omiljenim linijama (+5.0 bonus)"
- "Kupljeno 2x (+2.0 bonus)"
- "Jaka preporuka - X sličnih korisnika koristi ovu liniju"
- "Početna preporuka - još nema dovoljno podataka"
- "Popularna linija - često korišćena"

Ovo zadovoljava uslov da recommender objašnjava "zašto" je nešto preporučeno.

5. Putanje koda

Backend:

- backend/Services/RecommendationService.cs
  - GetRecommendedLinesAsync(...) (signali + računanje skora)
  - GetScoreExplanation(...) (objašnjenje preporuke)
- backend/Controllers/RecommendationsController.cs (endpointi)
- backend/Models/RecommendationFeedback.cs (model feedbacka)

Mobile:

- user-mobile/lib/screens/recommended_lines_screen.dart (UI prikaz + dialog objašnjenja + opis kako sistem radi)
- user-mobile/lib/services/recommendation_service.dart (API pozivi)

U nastavku su screenshotovi koda i aplikacije kao dokaz implementacije:

1. *Dio gdje se učitavaju Tickets, FavoriteLines, RecommendationFeedbacks (pozitivni i negativni)*

```
16      public async Task<List<RecommendedLineDto>> GetRecommendedLinesAsync(int userId, int count = 3)
17      {
18          var activeLines = await _context.TransportLines
19              .Include(tl => tl.TransportType)
20              .Include(tl => tl.Routes)
21              .Where(tl => tl.IsActive)
22              .ToListAsync();
23
24          if (!activeLines.Any())
25          {
26              return new List<RecommendedLineDto>();
27          }
28
29          var threeMonthsAgo = DateTime.UtcNow.AddMonths(-3);
30
31          var userTickets = await _context.Tickets
32              .Include(t => t.Route)
33                  .ThenInclude(r => r!.TransportLine)
34              .Where(t => t.UserId == userId && t.PurchasedAt >= threeMonthsAgo)
35              .ToListAsync();
36
37          var userFavorites = await _context.FavoriteLines
38              .Where(f => f.UserId == userId)
39              .Select(f => f.TransportLineId)
40              .ToListAsync();
41
42          var userNegativeFeedbackList = await _context.RecommendationFeedbacks
43              .Where(rf => rf.UserId == userId && !rf.IsUseful)
44              .Select(rf => rf.TransportLineId)
45              .ToListAsync();
```

Slika 1.1 Učitavanje aktivnih linija i korisničkih signala (karte, omiljene, feedback)

```
46          var userNegativeFeedback = userNegativeFeedbackList.ToHashSet();
47
48          var userPositiveFeedbackList = await _context.RecommendationFeedbacks
49              .Where(rf => rf.UserId == userId && rf.IsUseful)
50              .Select(rf => rf.TransportLineId)
51              .ToListAsync();
52          var userPositiveFeedback = userPositiveFeedbackList.ToHashSet();
53
54          var userLines = new HashSet<int>();
55          var userPurchasedLines = new Dictionary<int, int>();
56          foreach (var ticket in userTickets)
57          {
58              if (ticket.Route != null)
59              {
60                  var lineId = ticket.Route.TransportLineId;
61                  userLines.Add(lineId);
62                  if (!userPurchasedLines.ContainsKey(lineId))
63                  {
64                      userPurchasedLines[lineId] = 0;
65                  }
66                  userPurchasedLines[lineId]++;
67              }
68          }
69          foreach (var favoriteId in userFavorites)
70          {
71              userLines.Add(favoriteId);
72          }
```

Slika 1.2 Formiranje korisničkog profila: skup linija i broj kupovina po liniji

```
73
74          if (!userLines.Any() && !userPositiveFeedback.Any())
75          {
76              var fallbackLines = activeLines
77                  .Where(tl => !userNegativeFeedback.Contains(tl.Id))
78                  .Take(count)
79                  .Select(tl => new RecommendedLineDto
80                  {
81                      Id = tl.Id,
82                      LineNumber = tl.LineNumber,
83                      Name = tl.Name,
84                      Origin = tl.Routes.FirstOrDefault()?.Origin ?? string.Empty,
85                      Destination = tl.Routes.FirstOrDefault()?.Destination ?? string.Empty,
86                      TransportTypeName = tl.TransportType?.Name ?? string.Empty,
87                      IsActive = tl.IsActive,
88                      Score = 0.0,
89                      ScoreExplanation = "Početna preporuka - još nema dovoljno podataka"
90                  })
91                  .ToList();
92              return fallbackLines;
93          }
```

Slika 1.3 Cold-start fallback: početne preporuke kada nema dovoljno podataka

2. *Dio gdje se računa skor (kupovine, omiljene, pozitivan feedback, slični korisnici)*

```
95          var allTickets = await _context.Tickets
96              .Include(t => t.Route)
97                  .ThenInclude(r => r!.TransportLine)
98              .Where(t => t.PurchasedAt >= threeMonthsAgo && t.Route != null)
99              .ToListAsync();
100
101         var allFavorites = await _context.FavoriteLines
102             .ToListAsync();
103
104         var allPositiveFeedback = await _context.RecommendationFeedbacks
105             .Where(rf => rf.IsUseful)
106             .ToListAsync();
107
108         var userLineRatings = new Dictionary<int, Dictionary<int, double>>();
109
110         foreach (var ticket in allTickets)
111         {
112             if (ticket.Route == null) continue;
113
114             var uid = ticket.UserId;
115             var lid = ticket.Route.TransportLineId;
116
117             if (!userLineRatings.ContainsKey(uid))
118             {
119                 userLineRatings[uid] = new Dictionary<int, double>();
120             }
121
122             if (!userLineRatings[uid].ContainsKey(lid))
123             {
124                 userLineRatings[uid][lid] = 0;
125             }
126
127             userLineRatings[uid][lid] += 1.0;
128         }
```

Slika 2.1 Formiranje ratinga za sve korisnike

```
130         foreach (var favorite in allFavorites)
131         {
132             if (!userLineRatings.ContainsKey(favorite.UserId))
133             {
134                 userLineRatings[favorite.UserId] = new Dictionary<int, double>();
135             }
136 
137             if (!userLineRatings[favorite.UserId].ContainsKey(favorite.TransportLineId))
138             {
139                 userLineRatings[favorite.UserId][favorite.TransportLineId] = 0;
140             }
141 
142             userLineRatings[favorite.UserId][favorite.TransportLineId] += 3.0;
143         }
144 
145         foreach (var feedback in allPositiveFeedback)
146         {
147             if (!userLineRatings.ContainsKey(feedback.UserId))
148             {
149                 userLineRatings[feedback.UserId] = new Dictionary<int, double>();
150             }
151 
152             if (!userLineRatings[feedback.UserId].ContainsKey(feedback.TransportLineId))
153             {
154                 userLineRatings[feedback.UserId][feedback.TransportLineId] = 0;
155             }
156 
157             userLineRatings[feedback.UserId][feedback.TransportLineId] += 2.0;
158         }
```

Slika 2.2 Dodavanje omiljenih u rating i dodavanje pozitivnog feedbacka u rating

```
160         var userUserLines = userLineRatings.ToDictionary(
161             kvp => kvp.Key,
162             kvp => kvp.Value.Keys.ToHashSet()
163         );
164 
165         var userUserRatings = userLineRatings;
166 
167         var userUserLinesSet = userLines;
168 
169         var similarities = new Dictionary<int, double>();
170 
171         foreach (var otherUser in userUserLines.Keys)
172         {
173             if (otherUser == userId) continue;
174 
175             var otherUserLines = userUserLines[otherUser];
176             var intersection = userUserLinesSet.Intersect(otherUserLines).Count();
177             var union = userUserLinesSet.Union(otherUserLines).Count();
178 
179             if (union == 0) continue;
180 
181             var jaccard = (double)intersection / union;
182             if (jaccard > 0)
183             {
184                 similarities[otherUser] = jaccard;
185             }
186         }
187 
```

Slika 2.3 Računanje sličnosti korisnika (Jaccard)

```
192        foreach (var line in activeLines)
193        {
194            if (userNegativeFeedback.Contains(line.Id)) continue;
195
196            var score = 0.0;
197
198            if (userPurchasedLines.ContainsKey(line.Id))
199            {
200                score += userPurchasedLines[line.Id] * 1.0;
201            }
202
203            if (userFavorites.Contains(line.Id))
204            {
205                score += 5.0;
206            }
207
208            if (userPositiveFeedback.Contains(line.Id))
209            {
210                score += 10.0;
211            }
212
213            foreach (var similarUser in similarities.Keys)
214            {
215                if (!userUserRatings.ContainsKey(similarUser)) continue;
216                if (!userUserRatings[similarUser].ContainsKey(line.Id)) continue;
217
218                var similarity = similarities[similarUser];
219                var rating = userUserRatings[similarUser][line.Id];
220                score += similarity * rating;
221            }
222
223            if (score > 0)
224            {
225                lineScores[line.Id] = score;
226            }
227        }
```

Slika 2.4 Računanje skora po liniji

```
229        var recommendedWithScores = lineScores
230            .OrderByDescending(kvp => kvp.Value)
231            .Take(count)
232            .ToList();
233
234        var recommendedLineIds = recommendedWithScores.Select(kvp => kvp.Key).ToList();
235        var scoreMap = recommendedWithScores.ToDictionary(kvp => kvp.Key, kvp => kvp.Value);
236
237        if (recommendedLineIds.Count < count && similarities.Any())
238        {
239            var popularLines = await _context.Tickets
240                .Include(t => t.Route)
241                    .ThenInclude(r => r!.TransportLine)
242                .Where(t => t.PurchasedAt >= threeMonthsAgo && t.Route != null)
243                .GroupBy(t => t.Route!.TransportLineId)
244                .Select(g => new { LineId = g.Key, Count = g.Count() })
245                .OrderByDescending(x => x.Count)
246                .Take(count - recommendedLineIds.Count)
247                .Select(x => x.LineId)
248                .ToListAsync();
249
250            foreach (var popularLineId in popularLines)
251            {
252                if (!recommendedLineIds.Contains(popularLineId) &&
253                    !userNegativeFeedback.Contains(popularLineId))
254                {
255                    recommendedLineIds.Add(popularLineId);
256                    scoreMap[popularLineId] = 0.1;
257                }
258            }
259        }
260
```

```
261         if (recommendedLineIds.Count < count)
262         {
263             var additionalLines = activeLines
264                 .Where(tl => !userNegativeFeedback.Contains(tl.Id) &&
265                              !recommendedLineIds.Contains(tl.Id))
266                 .Take(count - recommendedLineIds.Count)
267                 .Select(tl => tl.Id)
268                 .ToList();
269
270             foreach (var additionalLineId in additionalLines)
271             {
272                 recommendedLineIds.Add(additionalLineId);
273                 scoreMap[additionalLineId] = 0.0;
274             }
275         }
```

Slika 2.5_1 i Slika 2.5_2 Odabir top preporuka i fallback popularne/dodatne linije

```
277         var recommendedLines = activeLines
278             .Where(tl => recommendedLineIds.Contains(tl.Id))
279             .OrderByDescending(tl => scoreMap.ContainsKey(tl.Id) ? scoreMap[tl.Id] : 0.0)
280             .Select(tl =>
281             {
282                 var score = scoreMap.ContainsKey(tl.Id) ? scoreMap[tl.Id] : 0.0;
283                 var purchasedCount = userPurchasedLines.ContainsKey(tl.Id) ? userPurchasedLines[tl.Id] : 0;
284                 var explanation = GetScoreExplanation(tl.Id, score, userPositiveFeedback.Contains(tl.Id), userFavorites.Contains(tl.Id), purchasedCount, similarities.Count);
285
286                 var hasPositive = userPositiveFeedback.Contains(tl.Id);
287                 var hasNegative = userNegativeFeedback.Contains(tl.Id);
288
289                 return new RecommendedLineDto
290                 {
291                     Id = tl.Id,
292                     LineNumber = tl.LineNumber,
293                     Name = tl.Name,
294                     Origin = tl.Routes.FirstOrDefault()?.Origin ?? string.Empty,
295                     Destination = tl.Routes.FirstOrDefault()?.Destination ?? string.Empty,
296                     TransportTypeName = tl.TransportType?.Name ?? string.Empty,
297                     IsActive = tl.IsActive,
298                     Score = Math.Round(score, 2),
299                     ScoreExplanation = explanation,
300                     HasPositiveFeedback = hasPositive ? true : null,
301                     HasNegativeFeedback = hasNegative ? true : null
302                 };
303             })
304             .ToList();
305
306         return recommendedLines;
```

Slika 2.6 Kreiranje response-a i objašnjenje preporuke

3. *Funkcija GetScoreExplanation(...) (pravila za objašnjenje preporuke)*

```
350     private string GetScoreExplanation(int lineId, double score, bool hasPositiveFeedback, bool isFavorite, int purchasedCount, int similarUsersCount)
351     {
352         if (hasPositiveFeedback)
353         {
354             return $"Označili ste kao korisno (+10.0 bonus)";
355         }
356
357         if (isFavorite && purchasedCount > 0)
358         {
359             return $"U vašim omiljenim linijama i kupljeno {purchasedCount}x (+{5.0 + purchasedCount * 1.0} bonus)";
360         }
361
362         if (isFavorite)
363         {
364             return $"U vašim omiljenim linijama (+5.0 bonus)";
365         }
366
367         if (purchasedCount > 0)
368         {
369             return $"Kupljeno {purchasedCount}x (+{purchasedCount * 1.0} bonus)";
370         }
371
372         if (score >= 10.0)
373         {
374             return $"Izvrsna preporuka - visoka sličnost sa drugim korisnicima";
375         }
376
377         if (score >= 5.0)
378         {
379             return $"Jaka preporuka - {similarUsersCount} sličnih korisnika koristi ovu liniju";
380         }
```

```
381
382         if (score >= 2.0)
383         {
384             return $"Dobra preporuka - slični korisnici su kupili karte za ovu liniju";
385         }
386
387         if (score >= 1.0)
388         {
389             return $"Srednja preporuka - bazirana na sličnostima";
390         }
391
392         if (score >= 0.3)
393         {
394             return $"Početna preporuka - još nema dovoljno podataka";
395         }
396
397         if (score > 0.0)
398         {
399             return $"Popularna linija - često korišćena";
400         }
401
402         return "Dodatna preporuka - aktivna linija";
403     }
404 }
405
```

Slika 3.1 i 3.2 Objašnjenje preporuke (ScoreExplanation) na osnovu ponašanja korisnika i skora

4. *Prikaz dugmeta "Kako funkcioniše sistem preporuke?" i mjesta gdje se otvara dialog "Objašnjenje preporuke"*

```
221                     Padding(
222                       padding: const EdgeInsets.all(16.0),
223                       child: OutlinedButton.icon(
224                         onPressed: () {
225                           _showSystemExplanationDialog();
226                         },
227                         icon: Icon(Icons.info_outline, color: Colors.blue[700]),
228                         label: const Text('Kako funkcioniše sistem preporuke?'),
229                         style: OutlinedButton.styleFrom(
230                           foregroundColor: Colors.blue[700],
231                           side: BorderSide(color: Colors.blue[300]!),
232                           padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
233                         ),
234                       ),
235                     ),
```

```
99    void _showSystemExplanationDialog() {
100     showDialog(
101       context: context,
102       builder: (BuildContext context) {
103         return AlertDialog(
104           title: Row(
105             children: [
106               Icon(Icons.info_outline, color: Colors.blue[700], size: 20),
107               const SizedBox(width: 8),
108               Flexible(
109                 child: Text(
110                   'Kako funkcioniše sistem preporuke?',
111                   style: const TextStyle(fontSize: 16),
112                 ),
113               ),
114             ],
115           ),
116           content: SingleChildScrollView(
117             child: ConstrainedBox(
118               constraints: BoxConstraints(
119                 maxWidth: MediaQuery.of(context).size.width * 0.8,
120               ),
121               child: Column(
122                 crossAxisAlignment: CrossAxisAlignment.start,
123                 mainAxisSize: MainAxisSize.min,
124                 children: [
125                   Text(
126                     'Sistem analizira vaša prethodna putovanja, kupljene karte i omiljene linije. Na osnovu sličnosti sa drugim korisnicima, preporučuje linije koje bi mogle biti od in
127                     style: TextStyle(
128                       fontSize: 14,
129                       color: Colors.grey[800],
130                     ),
131                   ),
```

```
132                   const SizedBox(height: 16),
133                   Text(
134                     'Kako se računa skor:',
135                     style: TextStyle(
136                       fontSize: 15,
137                       fontWeight: FontWeight.bold,
138                       color: Colors.grey[900],
139                     ),
140                   ),
141                   const SizedBox(height: 8),
142                   Text(
143                     '• Kupovina karte: +1.0 po karti\n• Dodavanje u omiljene: +5.0\n• Pozitivan feedback: +10.0\n• Slični korisnici: skor se množi sa sličnošću',
144                     style: TextStyle(
145                       fontSize: 14,
146                       color: Colors.grey[800],
147                     ),
148                   ),
149                   const SizedBox(height: 12),
150                   Text(
151                     'Linije su sortirane po skoru - prva linija ima najveći skor.',
152                     style: TextStyle(
153                       fontSize: 13,
154                       fontStyle: FontStyle.italic,
155                       color: Colors.grey[700],
156                     ),
157                   ),
158                 ],
```

Slika 4.1, 4.2 i 4.3 Mobilni UI: objašnjenje rada recommendera (dugme i dialog)

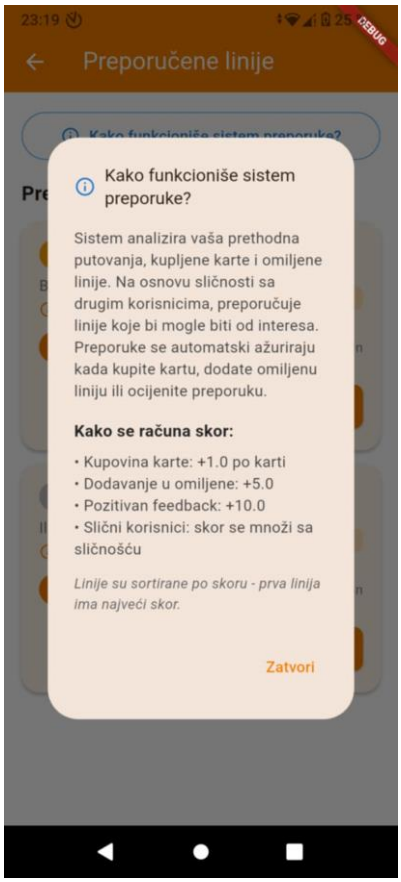
```
340                 ],
341               ),
342               const SizedBox(height: 4),
343               Text(
344                 '${line.origin} → ${line.destination}',
345                 style: TextStyle(
346                   fontSize: 14,
347                   color: Colors.grey[600],
348                 ),
349               ),
350               if (recommendedLine.scoreExplanation != null) ...[
351                 const SizedBox(height: 4),
352                 GestureDetector(
353                   onTap: () {
354                     _showExplanationDialog(recommendedLine.scoreExplanation!);
355                   },
356                   child: Icon(
357                     Icons.info_outline,
358                     size: 16,
359                     color: Colors.orange[700],
360                   ),
361                 ),
362               ],
363             ],
364           ),
```

Slika 4.4 Mobilni UI: objašnjenje preporuke po liniji (scoreExplanation)

5. Pokrenuta aplikacija: lista preporučenih linija i otvoren dialog 'Objašnjenje preporuke'

Slika 5.1 Pokrenuta aplikacija: preporučene linije i objašnjenje preporuke

Napomena: scoreExplanation prikazuje glavni (dominantni) razlog preporuke (npr. pozitivan feedback, omiljena linija ili broj kupovina). Ukupan skor može uključivati i dodatne doprinose (npr. sličnost sa drugim korisnicima), ali se u objašnjenju ne prikazuje kompletan breakdown.



Slika 5.2 Pokrenuta aplikacija: objašnjenje rada sistema preporuke

Preporuke se mijenjaju kada korisnik kupi kartu, doda omiljenu liniju ili ostavi feedback, jer se ti signali upisuju u bazu i koriste u scoringu.